

Life Orchestrator — Architecture Summary

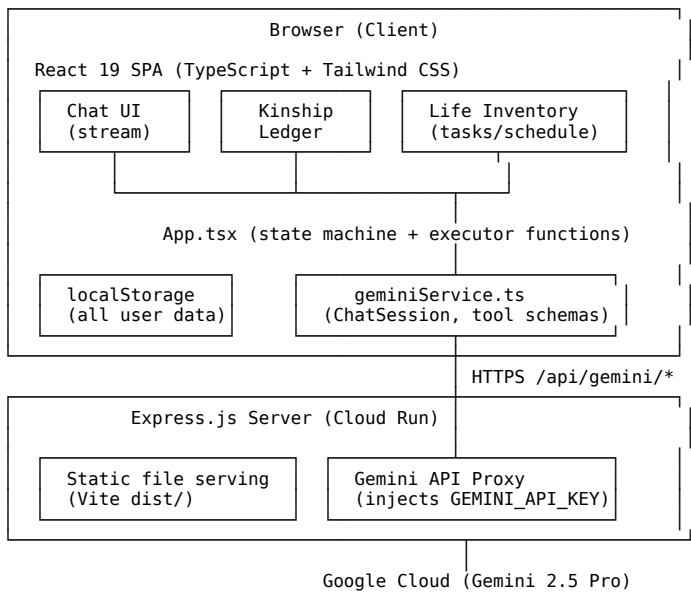
Purpose: A concise, interviewer-ready explanation of how Life Orchestrator is structured, why key decisions were made, and what engineering challenges were solved. Use this for system-design discussions, technical screens, and portfolio presentations.

Core Philosophy

Life Orchestrator is built on one central principle: **local-first, privacy-by-design**. No user data ever leaves the browser. No server-side database. No authentication system. The AI runs inference on Google's servers, but conversations are ephemeral — Google does not retain them.

This principle drives every major architectural decision in the project.

Architecture at a Glance



1. Frontend Architecture

View State Machine

Navigation is handled by a single-level state variable (`activeView`) rather than a router. This eliminates route configuration overhead and keeps all view logic in one place.

```
'chat' | 'ledger' | 'inventory' | 'storage' | 'settings' | 'legal'
```

Component Design Pattern: Inline Editing

All data-entry components (Kinship Ledger contacts, Inventory tasks) use click-to-edit with auto-save on `onBlur`. No separate "edit mode" or modal dialogs. This reduces cognitive overhead and matches spreadsheet-style UX expectations.

Optimistic UI + Async Confirmation

Actions (add task, log check-in) update UI immediately. AI confirmation and orchestration proposals arrive asynchronously — typically 3–6 seconds later as a background message. This eliminated multi-second blocking that previously froze the chat.

2. AI Integration Architecture

Prompt Engineering as Application Logic

The AI's behavior is defined entirely by a structured system prompt (`SYSTEM_INSTRUCTION` in `constants.ts`). Changing AI behavior is a prompt edit, not a code refactor. This is a deliberate engineering choice:

- Nuanced priority reasoning is difficult to express as deterministic code
- Prompt-driven behavior is easier to test and iterate
- The prompt IS the documentation of AI orchestration rules

Temporal Mode System

The AI operates in one of three modes based on the currently viewed date:

Mode	When	AI Behavior
Reflection	Past dates	Retrospective analysis, no orchestration allowed
Active	Today	Full capabilities, real-time optimization
Planning	Future dates	Tentative scheduling, contingency planning

Each mode uses a distinct system instruction appended to the base prompt. This prevents logical impossibilities (e.g., scheduling something yesterday) and ensures tone-appropriate communication.

Function Calling as Safety Boundary

The AI cannot execute arbitrary code. It communicates intent by calling pre-defined TypeScript executor functions with validated JSON schemas. Even if the model produces unexpected output, the executor layer enforces:

1. Gemini schema validation (type checking at the API level)
2. TypeScript compile-time checks
3. Runtime validation in each executor
4. `localStorage` isolation (no file system or network access)

This is a defense-in-depth security design.

Streaming Architecture

AI responses render token-by-token using `generateContentStream`. A custom `withTimeout()` wrapper enforces:

- 30-second stream completion timeout
- 30-second inter-chunk timeout
- 60-second timeout for full orchestration streams

An `AbortController` allows in-flight streams to be cancelled immediately when the user switches dates or starts a new message.

3. Data Architecture

Fixed vs. Flexible Task Dichotomy

Tasks are classified as one of two types:

- **Fixed:** Hard-scheduled appointments (meetings, interviews, PT sessions). These anchor the day — the AI cannot move them.
- **Flexible:** Optimizable time-blocks (gym, email, meal prep). The AI fills these into optimal energy windows.

This binary classification drives the core orchestration algorithm and avoids the ambiguity of a pure priority scale.

Kinship Debt Algorithm

Relationship health is quantified by a simple formula:

`Kinship Debt = Priority × Days Since Last Contact`

< 5 → Stable
5–10 → Needs Attention
10–20 → Critical
> 20 → Overdue

This makes relationship neglect objectively measurable. High-priority relationships (score 9–10) trigger alerts after just 1–2 days without contact.

Recurrence Storage Strategy

Recurring tasks store a single rule rather than N instances:

```
{ frequency: 'weekly', weekdays: [1, 3] } // Every Monday and Wednesday
```

Client-side expansion happens on-demand per date. This saves ~98% storage and makes bulk edits trivial.

Memory System

The `Memory[]` array in `localStorage` gives the AI persistent context across sessions. Three types:

- **preference** — user behavior patterns
- **decision** — strategic choices
- **fact** — important long-term context

A FIFO limit of 100 memories prevents unbounded context window growth.

4. Storage Architecture

Problem: 5 MB Hard Limit

Browser `localStorage` has a ~5 MB per-origin limit. Hitting it silently breaks the app.

Mitigation Strategy

1. **Real-time quota display** — percentage bar always visible
2. **Warnings** — amber at 80%, red at 90%
3. **Granular cleanup** — delete by date, not nuclear reset
4. **Image compression** — 800×800 px, JPEG 70% quality $\approx$ 100 KB per image
5. **Upload limits** — 5 images/day maximum
6. **Base64 accounting** — storage usage formula includes $\times 1.33$ overhead multiplier

Future Migration Path

`services/db.ts` contains a complete IndexedDB schema. The migration from `localStorage` to IndexedDB is prepared but not yet activated. Triggers for migration:

- User hits 5 MB quota regularly
 - Vector embeddings feature launches
 - Multi-device sync is implemented
-

5. Backend Architecture

Why a Server at All?

The API key for Gemini must not be exposed in client-side JavaScript bundles. The Express server acts as a thin proxy that:

1. Receives requests at `/api/gemini/*` with a placeholder key
2. Strips the placeholder and injects the real `GEMINI_API_KEY` from Cloud Run env vars
3. Forwards to Google's Generative Language API with streaming preserved

This is the only server-side logic. The server has zero business logic.

Static Hosting

The same Express server serves the Vite-built SPA (`dist/`) with:

- **Cross-Origin-Opener-Policy**: `same-origin-allow-popups` — required for Google OAuth popup
 - SPA fallback `app.get('*')` — all unknown routes serve `index.html` for client routing
-

6. Deployment Architecture

Multi-Stage Docker Build

Stage 1 (build image) Node 22 Alpine → npm ci → vite build → creates dist/
Stage 2 (run image) Node 22 Alpine → copy dist/ + server code → npm install --omit=dev → node index.js

The final image contains only the Express server and the built static files. No TypeScript compiler, no Vite, no devDependencies.

Google Cloud Run

- Serverless container execution — no VM management
- Auto-scales to zero when idle (zero cost when unused)
- GEMINI_API_KEY supplied as Cloud Run environment variable — never in source code
- Deployment automated via advanced_deploy.ps1 PowerShell script

7. Key Engineering Decisions Summary

Decision	Choice Made	Alternative Rejected	Reason
Data storage	localStorage only	Server-side database	Privacy by design; zero server cost
Navigation	View state machine	React Router	Sufficient at single-level nav scale
State management	Props + local state	Redux / Zustand	No cross-cutting state needs
Styling	Tailwind CSS	CSS-in-JS	No runtime overhead; atomic reuse
AI behavior	System prompt	Hardcoded algorithms	Flexibility, iterability, readability
Task classification	Fixed / Flexible binary	Priority 1–10 scale	Binary captures "movability" clearly
Server role	API key proxy only	Full BFF	Minimal attack surface; privacy
Dependency policy	Minimal, justified	Preemptive abstractions	Add when pain justifies complexity

Generated for portfolio and job application use. Keep in sync with major architecture changes.